



Making Research Software Citable & Discoverable

UC Carpentries Workshop Series · Fall 2026

Tim Dennis
UCLA OSPO

Leigh Phan
UCLA OSPO

Karla Padilla
UCSD OSPO

Reid Otsuji
UCSD OSPO

Laura Langdon
UC OSPO

2026-09-23

Welcome!

UC Carpentries Workshop Series · Fall 2026

We'll begin shortly. Grab the collaborative notes and settle in.

Etherpad: pad.carpentries.org/2026-fall-uc-sharing-research-software

Lesson site: ucospo.net/research-software-citable-discoverable

Code of Conduct

We agree to

- Use **welcoming and inclusive** language
- Be **respectful** of different viewpoints and experiences
- Accept **constructive criticism** gracefully
- Show **courtesy and respect** toward everyone here

If something's wrong

- Tell an **instructor** or the **host** — chat DM or email, privately
- Or report to the **CoC Committee**:
coc@carpentries.org
- All reports are **kept confidential** and taken seriously

Full text & report form: docs.carpentries.org/policies/coc

How we learn together

Our pedagogy

- **Hands-on:** you do each step in your own fork as we go
- **Connect to what you know:** new skills land faster that way
- **Fewer concepts, done well:** not a race

Managing expectations

- **Usable today:** practical skills, not full coverage
- **Group pace:** your questions set the speed
- **Mistakes are expected:** for all of us

Software Is the Missing Scholarly Output

The question that starts this lesson

“The journal accepted my paper, but they want a citable version of the code, with a DOI. How do I do that?”

Ten years ago the same question arrived about **datasets**, and libraries built the answer: a persistent identifier, a catalog record, a citation format, a preservation copy. Software mostly hasn't gotten that treatment yet.

You write research software, and want to make your own code citable, reusable, and discoverable.

You support people who do, as a librarian, OSPO member, or research-support specialist.

Supporting others slides translate each step into consultation work.

How common is the software citation gap?

59.5%

of 3.2 million PubMed Central articles mention at least one piece of software

38%

of 2020 articles give enough detail to **uniquely identify** the software they used

Both figures: Schindler et al. (2022), PeerJ Computer Science, doi:10.7717/peerj-cs.835

A GitHub URL isn't a citation

The wrong way

“We used the analysis script from `https://github.com/UC-OSP0-Network/software-demo` ”

- URLs break (link rot)
- No version specified — which run produced which result?
- No author name, so no credit and no citation counts

The right way

Dennis, T., Phan, L., Otsuji, R., & Padilla, K. (2026). *Biodiversity Analysis Toolkit* (v0.1.0). Zenodo. <https://doi.org/10.5281/zenodo.123456>

- The DOI survives repo moves and even GitHub going away
- The version is explicit, so someone can rerun the exact code
- Authors get credit and citation counts

You'll create both pieces today: a `CITATION.cff` file and a DOI.

Software has a durability problem

The Bus Factor

The number of people who need to be “hit by a bus” before a project becomes unmaintainable.
For most research code, the answer is one.

Critical infrastructure too: before 2014, OpenSSL secured about a sixth of the web’s encrypted traffic on **~\$2,000/year** in donations, with **one person paid** to maintain it. Then Heartbleed happened.

Core Infrastructure Initiative, Linux Foundation. en.wikipedia.org/wiki/Core_Infrastructure_Initiative

The fix: FAIR4RS principles

F — Findable

DOI, CITATION.cff, metadata

A — Accessible

Public GitHub, archived on Zenodo

I — Interoperable

Standard formats, documented dependencies

R — Reusable

LICENSE, README, environment files

Before → after

Before (main)

- ❌ No LICENSE
- ❌ No citation information
- ❌ No DOI
- ❌ Minimal README
- ❌ No community or metadata files

After (05-add-metadata)

- ✅ LICENSE (BSD-3-Clause)
- ✅ CITATION.cff
- ✅ Complete README
- ✅ CONTRIBUTING.md, CODE_OF_CONDUCT.md
- ✅ .zenodo.json, DOI badge

This is what makes software Findable, Accessible, Interoperable, and Reusable.

Real-world example: Spack

Used by national labs, universities, and supercomputing centers: github.com/spack/spack

-  README with clear description and getting-started guide
-  LICENSE (Apache-2.0 or MIT, your choice)
-  CITATION.cff file
-  Documentation
-  Code of conduct and contributing guide

What we'll build together

Starting from the bare `main` state in your fork, progressively:

1. Sharing research software (*exploring the starting repository*)
2. Adding an open-source license
3. Creating a CITATION.cff file
4. Minting a DOI and creating a release
5. Improving metadata and discoverability
6. Wrap-up and reflection

View-only reference branches (`02-add-license` , `03-add-citation` , `04-create-release` , `05-add-metadata`) let you check your work at any point.

Sharing Research Software Effectively

Present, but not yet usable

Most research software on GitHub is an unprocessed accession: it made it into the building but not into the collection. Technically held, practically invisible.

Public is not the same as shared. A repo with no license, no citation, and no real description is an unlocked reading room with no catalog.

Fork the demo repository

Every hands-on step happens in **your own fork**, not the shared demo repo.

1. Open `github.com/UC-OSP0-Network/software-demo`
2. Click **Fork** → **Create fork** (defaults are fine — `main` branch only)
3. Confirm the top-left reads `YOUR-USERNAME/software-demo` , not `UC-OSP0-Network/software-demo`

Reference branches are there to check your work, never to build on. Full setup details: ucospo.net/research-software-citable-discoverable/learners/setup

The starting point

```
software-demo/  
├── README.md  
├── requirements.txt  
└── src/  
    └── analysis.py
```

Deliberately bare. By the end it will have a license, citation metadata, a DOI, and a real README.

Supporting others: a four-question triage

When a researcher brings you their repo, do a fast triage, not a code review.

1. **Rights** — is there a license? Can this legally be reused at all?
2. **Credit** — can a stranger tell who made it and how to cite it?
3. **Runnable** — is there any record of what it takes to run?
4. **Findable** — would anyone outside the lab ever discover it?

The goal is the **single highest-value next step**, not fixing everything.

Try it now (2 min): answer all four for your fork of `software-demo` .

Key points

- A GitHub URL is not a citation. A DOI is permanent and version-specific.
- Software that isn't deposited anywhere depends on one person and one platform
- FAIR4RS (Findable, Accessible, Interoperable, Reusable) is the framework
- Public is not the same as shared: license, citation, and description do the work

Up next

Karla Padilla — Adding an Open-Source License

Laura Langdon — Creating a CITATION.cff File

Reid Otsuji — Minting a DOI and Creating a Release

Then back to Tim for Improving Metadata and Discoverability.